

TP1 : Introduction à SOA & SOAP	A.U. : 2025/2026
Matière : SoA et Microservices Classe : 4Info Enseignant : Dr. Salah Gontara	

Objectifs

- Comprendre l'idée **SOA** : service, contrat, client, interopérabilité.
- Comprendre la structure d'un message **SOAP** : *Envelope*, *Header*, *Body*.
- Lire un **WSDL** : definitions, types, message, portType, binding, service.
- Tester une requête SOAP via **Postman** et interpréter la réponse et un **SOAP Fault**.

Prérequis

- **Postman** installé.
- Accès Internet (service SOAP public).

Service utilisé dans ce TP (Hello SOAP)

Endpoint (POST) : <https://apps.learnwebservices.com/services/hello>

WSDL : <https://apps.learnwebservices.com/services/hello?WSDL>

Partie A — SOA & lecture du WSDL (le contrat)

A1. Rappel SOA

- Un **service** expose une fonctionnalité via un **contrat** (WSDL).
- Un **client** consomme le service en respectant le contrat (sans connaître l'implémentation interne).

A2. Exploration du WSDL

Ouvrir le WSDL : <https://apps.learnwebservices.com/services/hello?WSDL>

Travail demandé : repérer et expliquer le rôle des éléments suivants :

- `<wsdl:definitions>` : racine du contrat, *targetNamespace*.
- `<wsdl:types>` : schéma XSD (définition des structures échangées).
- `<wsdl:message>` : messages d'entrée/sortie (référence vers des éléments XSD).
- `<wsdl:portType>` : interface abstraite (opérations + input/output).
- `<wsdl:binding>` : détails concrets (SOAP/HTTP, style document, use literal).
- `<wsdl:service>` + `<soap:address>` : URL réelle (endpoint) à appeler.

A3. Questions

1. Où le WSDL donne-t-il l'**URL** du service ?
2. Où sont définis les champs **Name** (entrée) et **Message** (sortie) ?

3. Quelle différence entre `portType` et `binding` ?
4. Quel est le nom de l'opération exposée par ce service ?

Partie B — SOAP Envelope & appel avec Postman

B1. Structure d'une SOAP Envelope

- **Envelope** : racine du message SOAP (wrapper).
- **Header** (optionnel) : métadonnées (sécurité, routage, etc.).
- **Body** (obligatoire) : payload métier (requête/réponse).

B2. Configurer Postman (POST SOAP)

Dans Postman, créer une requête :

- Méthode : **POST**
- URL : `https://apps.learnwebservices.com/services/hello`
- Headers :
 - Content-Type: `text/xml; charset=UTF-8`
 - (optionnel) SOAPAction: `"` (souvent non requis ici)
- Body : **raw** → **XML**

B3. Requête SOAP (dans Postman)

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header/>
  <soapenv:Body>
    <HelloRequest xmlns="http://learnwebservices.com/services/hello">
      <Name>VotreNom</Name>
    </HelloRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

B4. Réponse attendue

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <HelloResponse xmlns="http://learnwebservices.com/services/hello">
      <Message>Hello VotreNom!</Message>
    </HelloResponse>
  </soap:Body>
</soap:Envelope>
```

B5. Questions

1. Dans la réponse, où se trouve le **payload métier** ?
2. Quel tag contient le message final ?
3. Indiquer ce qui relève de la **structure SOAP** vs des **données métier**.

Partie C — Provoquer et analyser un SOAP Fault

C1. Provoquer une erreur

Modifier volontairement la requête (au choix) :

- Casser une balise (ex : `</Name>` → `</Name`)
- Supprimer une fermeture de tag, ou supprimer `<soapenv:Body>` (message invalide)

Puis cliquer **Send**.

C2. Observation

Vous devez observer une réponse d'erreur contenant typiquement :

- `<soap:Fault>`
- `faultcode`
- `faultstring`

C3. Questions

1. Relever `faultcode` et dire s'il s'agit d'une erreur côté client ou serveur.
2. Relever `faultstring` et expliquer l'origine probable de l'erreur.
3. Où se trouve le **Fault** dans la structure SOAP (Header ou Body) ?